

1. Core Assignment Objectives

The main purpose of this assignment is to convert your existing Next.js web application into an installable **Progressive Web App (PWA)** that supports offline capabilities and custom visual themes. You will need to implement, test, and document all the practical lab steps outlined in the module.

2. Detailed Task Breakdown

Task 1: Create a Detailed Practical Report (Documentation)

You must create a comprehensive step-by-step document that records your setup, implementation, and application testing.

- **What to Include:**
 - **Code Modifications:** Clear explanations of any code segments you modified or added to enable PWA features.
 - **Screenshots:** High-quality screenshots showing your actual code snippets, terminal commands, and how the application behaves in the browser.
 - **Workflow Descriptions:** Clear explanations detailing the operational workflow of your setup.
- **File Requirements:** * The completed report **must be saved in PDF format**.
 - You must save and store this PDF file directly **inside your web project folder directory**.

Task 2: Upload to GitHub & Deploy

You need to commit your project workspace to GitHub. Ensure you fulfill the following explicit naming conventions:

- **Repository Name:** **PWL MICROSERVICES-NPM** (Note: Replace **NPM** with your actual Student Identification Number).
- **Commit Message:** Must be exactly **"Praktikum 7"**.

Deadline

- The assignment is due exactly **1 week from today** (aligned with your practical class timetable).

3. Roadmap of Implementation Steps You Need to Document

To get full marks on your report, you should follow and screenshot the **8 major steps** outlined in the lab module:

1. **Run the Base Application:** Ensure your previous Next.js project is fully functional using **npm run dev**.
2. **Install PWA Dependencies:** Run **npm install next-pwa** to install the service worker configuration package. (Take a screenshot of the terminal installation).

3. **Create the Web App Manifest:** Create a `manifest.json` file in your root project containing metadata like `name`, `short_name`, `icons`, `start_url`, and `display: "standalone"`. *(Explain what these properties mean in your report).*
4. **Add Asset Icons:** Put application icons matching various sizes (from 72x72 up to 512x512 pixels) inside the `public/icons/` folder. These are used for your mobile launcher and app splash screens.
5. **Register Manifest in Layout:** Inject the manifest file into your Next.js application by importing/defining metadata inside `app/layout.tsx`.
6. **Configure the Service Worker:** Set up the `next-pwa` plugin options inside your `next.config.ts` (or `.js`) file to automate background caching and offline modes.
7. **Run a Production Build:** Because caching logic generally does not run during development to prevent debugging conflicts, you must build and launch the production application:
8. Bash

`npm run build`

`npm start`

- 9.
10. *(Include a screenshot of a successful, error-free build compilation).*
11. **Perform Caching and Mobile Testing:** Open your browser's Developer Tools (Inspect Element $\rightarrow$ **Application** tab $\rightarrow$ **Service Workers**). Test and document the following behaviors:
 - **Cache First Strategy:** Loading static files directly from local storage.
 - **Network First Strategy:** Attempting to query fresh data from the server first, falling back to cache if offline.
 - **Stale-While-Revalidate Strategy:** Serving old cache immediately while quietly updating data in the background. Document what happens when shifting pages while completely offline (e.g., encountering an expected Error `106/ERR_INTERNET_DISCONNECTED`) and how background sync fixes it when returning online.
 - **Mobile Display / Installation:** Show screenshots of either a mobile emulator or physical device prompt demonstrating the **"Add to Home Screen"** feature launching your web app as a standalone application